# Catalan bijections facilitator guide

Library slot 12. Key for W12-K-v4, W12-23-v3 and W12-45-v4. Each student packet has Problems 1-6, one per page, except that K-1 Problem 4 continues on page 5, so the K-1 packet has seven pages. Draft and unpiloted; the slot number does not set a meeting calendar.

## The mathematical facts

Three equivalent objects. For each nonnegative integer n, these are in bijection: complete noncrossing pairings of 2n fixed points around a circle, with strings inside it (or in a row, with arcs above it); paths of n up-right steps U and n down-right steps D that start and end at height zero and never go below it; and finite ordered rooted trees with n edges. A tree is connected and has no cycles; the children at every vertex have a fixed left-to-right order. Circle labels stay fixed: rotations or reflections do not identify different partner lists.

The maps are reversible. Read pairing endpoints in order, writing U at a pair’s first end and D at its other end. At each D, the inverse closes the most recent unclosed U; the noncrossing rule forces this choice. For a tree, visit whole child branches left-to-right, writing U when moving away from the root and D when returning. Its inverse adds a new rightmost child at U and moves to the parent at D. These rules prove the bijections; equal counts alone do not.

The counts. Let C(n) count any one of these sets and let C(0)=1. For n at least 1, C(n) is the sum of C(i)C(n-1-i) for i from 0 to n-1. The unique first-return split U P D Q proves this recurrence. Thus C(0),...,C(6) = 1, 1, 2, 5, 14, 42, 132. The full split argument is on page 9.

Parity and neighbors. An odd number of points cannot be completely paired. On a circle, a fixed chord can be completed without crossings exactly when it leaves an even number of other points on each side. Every nonempty complete noncrossing circle pairing contains a pair of circular neighbors. The proofs and exact instances follow on pages 3-4.

## Where the worksheets investigate these facts

| Band and problems | What children investigate or explain |
|---|---|
| K-1 P1-P2 | Find the five six-dot pairings, then justify completeness. The two four-dot cases are printed examples. |
| K-1 P3-P6 | Complete fixed chords; explore eight-dot pairings; test parity; decide whether neighbors can be avoided. |
| Grades 2-3 P1-P4 | Enumerate six-dot pairings; encode paths; investigate the inverse; match all five paths and pairings. |
| Grades 2-3 P5-P6 | Decide which codes are legal; enumerate all 14 eight-dot pairings. |
| Grades 4-5 P1-P4 | Enumerate three-edge trees; encode and reconstruct trees; investigate uniqueness and characterize legal codes. |
| Grades 4-5 P5-P6 | Count trees with four, five and six edges; propose and justify a general counting rule. |

The examples introduce a representation or a walk, not a counting method. Let children conjecture before giving the facts above. Use drawings and explanations; keep hints and formal recurrence notation until needed.

## Preparation and entry choices

## What children are investigating

A complete collection can be organized by a forced choice. Pairings, up-and-down paths and ordered rooted trees carry the same information, but choose one route deeply before adding another representation. The core outcome is an explanation of why a collection is complete or why a code can be reversed.

## Prerequisites and entry choices

| Entry | Requirements and suitable destination |
|---|---|
| K-1 | Follow spoken rules; pair objects; distinguish fixed positions. No reading or arithmetic needed beyond small counts. Reach an exhaustive six-dot collection or an impossibility argument. |
| Grades 2-3 | Read left-to-right; track one-step rises/falls. Use drawing or U/D cards. Reach a reversible pairing/path code. |
| Grades 4-5 | Track branching order and nested instructions; organize small cases. Multiplication/addition needed only for the final counts. Reach the inverse tree construction or first-return recurrence. |

## Prepare for ten children and three adults

Print the desired pages single-sided on US Letter at 100%. Prepare 50 flexible strings about 20 cm long, 100 paper endpoint discs (15-20 mm), and 100 direction cards (50 U and 50 D, about 20 mm square), plus pencils, erasers and blank paper. These are proposed quantities, not verified inventory. Yarn is optional: pencil arcs are mathematically exact. Use the large mats for objects and the small diagrams only for pencil records. Test the maximum-size discs on a printed mat; labels must remain visible. Fold spare string outside the working region without making an extra route or crossing. Twenty centimeters covers the longest route, the arc from 1 to 6 on the grades 2-3 P1 row: its ends are 12 cm apart, and even a half circle is 19 cm. The longest chord on a K-1 circle is 8.6 cm. These lengths are measured from the printed pages, not rehearsed; before the meeting, lay one full eight-dot pairing in strings on K-1 page 3 to check that the tails fold away.

Keep the three stable adult-led tables. A child may use another band by readiness. Have each adult try one pairing and one code before the meeting. Do not hand out the answer galleries. Preserve fixed point numbers, no crossings, and left-to-right child order. Moving a string without changing partners makes the same pairing. Moving a branch left of another can change an ordered tree.

## A flexible one-hour menu

| Minutes | Use |
|---|---|
| 0-4 | Free handling of strings and discs; allow a few pairings before adding restrictions. |
| 4-8 | Shared launch below. |
| 8-25 | First sustained investigation: K-1 P1-P2; grades 2-3 P1-P2; grades 4-5 P1-P2. |
| 25-28 | Brief standing break: one step up for U, one down for D; stop before going below the starting level. |
| 28-50 | Choose depth: K-1 P3, then P5; middle P3-P5; upper P3-P4. K-1 P4 and P6, middle P6 and upper P5-P6 are surplus work. |
| 50-60 | Each table shares one construction and one reason. Save partial collections rather than require every page. |

## Shared launch in four minutes

Put four fixed discs around a circle. Ask: “Join every dot to one other dot, with strings inside the circle. Can you make a pairing?” After a legal attempt, slide a string while keeping its endpoints: “Did anyone get a new partner?” Make the crossing pairing and identify the forbidden crossing. Show both legal four-dot pairings, as on K-1 page 1; the six-dot collection remains for children to find. For the older table, add a separate two-edge tree and walk away from and back to its root; show only what U and D record.

## Hold hints until a child needs one

First ask the child to show their current attempt. Wait long enough for another attempt. Give the first hint below, then the next only if needed. A drawing or arrangement of strings is a real proof when it shows why all possibilities are covered. Do not insist on formal notation.

## K-1 exact answers and held hints

### Problem 1

The printed four-dot examples are 12|34 and 14|23; 13|24 crosses. The task is to find all five six-dot pairings in the gallery on page 6. Here 12 means the pair {1,2}; a vertical bar separates pairs. Hint: keep one dot in place and give it a different partner. Do not count a rotated drawing as the same unless every numbered dot still has the same partner.

### Problem 2

Check the P1 collection for repeats or omissions. Exactly five pairings: 12|34|56, 12|36|45, 14|23|56, 16|23|45, 16|25|34. To establish completeness, 1 can meet only 2, 4 or 6; these give 2, 1 and 2 completions. A chord leaves an even number of dots on each side. Hint ladder: “Where can 1 go?” then “Can a string escape from one side of that pair without crossing it?”

# K-1 continued

### Problem 3

Keep each supplied chord. For 14, completions are 14|23|56|78 and 14|23|58|67. For 16, they are 16|23|45|78 and 16|25|34|78. The 13 and 15 starts are impossible: they trap respectively one or three dots on one side, and a string cannot leave without crossing the fixed chord. Every start is printed twice, so the frames do not show which starts can be finished: the 14 and 16 frames hold the two completions of each, and the 13 and 15 frames hold none. Hint: point to the two regions cut by the fixed string and pair within them.

### Problem 4

There are 14 possible eight-dot pairings, listed and drawn on page 7. The task asks for as many as possible, so a well-organized partial collection is a valid stopping point. Group by 1’s partner: 2, 4, 6, 8 give 5, 2, 2, 5 completions. Pages 4 and 5 hold 16 frames for the 14 pairings, so the frames do not give away the count. Hint: two drawings are the same only if every dot has the same partner. Sort the drawings by 1’s partner, then compare partners dot by dot within each group. A match at one dot is not a repeat: 12|34|56|78 and 12|38|45|67 agree at dot 1 but are different.

### Problem 5

Ten, four and six dots are possible: join adjacent pairs (12,34,56,78,9-10 as far as the circle allows). Three, five and seven dots are impossible. Each string removes two unpaired dots, so an odd starting number always leaves an odd number; it cannot reach zero. A physical demonstration that repeatedly removes pairs is sufficient. Hint: “What happens to the number left each time you place one string?”

### Problem 6

No. Every noncrossing complete pairing has a pair of circular neighbors. Proof: consider any chord and one side of it. If there are dots on that side, they must be paired entirely within that side. Choose a pair with the fewest intervening dots in the linear reading order 1,...,8. If it had any, those dots would have a pair between them with fewer intervening dots, a contradiction. Thus some consecutive numbered pair is joined, and it is also a pair of circular neighbors. Hint ladder: look for an “innermost” string; then ask what could happen to a dot trapped underneath it.

## Common teaching traps

No overpasses or underpasses: a string lifted over another still represents a crossing. The noncrossing condition is essential. The four-point count would be three if crossings were allowed. Use fixed numbered positions; do not quotient by rotations or reflections. With trees, retain the order of branches but ignore line lengths and angles.

## Useful stopping evidence

K-1: a child can show all five six-dot cases or explain a trapped odd set. Middle: a child reconstructs a new pairing from a path and identifies why it is forced. Upper: a child reconstructs a tree, or separates an exhaustive count into disjoint cases. These are stronger evidence than a full packet.

# Grades 2-3 exact answers

### Problem 1

The ten-dot example is 14|23|5-10|67|89. The six-dot task has the same five pairings as K-1 P1-P2; use the page 6 gallery. Above a line, the endpoints still have fixed order. Completeness is the same 2+1+2 split by point 1’s partner. Hint: organize what you have before trying for another.

### Problem 2

Worked example: UDUUDDUD, from 12|36|45|78. For the three unanswered cases, top to bottom: UUUDDD, UUDDUD, UDUDUD. Their height sequences, including the start, are 0,1,2,3,2,1,0; 0,1,2,1,0,1,0; and 0,1,0,1,0,1,0. All stay at or above the starting level and end there. Height counts opened pairs that have not yet closed; that number cannot be negative. Hint: place one counter aside at each opening and remove one at each closing.

### Problem 3

Top to bottom: UUDUDD gives 16|23|45; UDUUDD gives 12|36|45; UUDDUUDD gives 14|23|58|67; UUUDUDDD gives 18|27|34|56. No path has two noncrossing inverse pairings. At every D, close the most recently opened unclosed U. If an older opening were closed first, the more recent opening would have to close later, causing interleaving endpoints and a crossing. This is the stack rule, and it is forced, not merely a convenient algorithm. Hint: try closing an older string and finish the remaining one.

### Problem 4

Exactly five paths; see page 6. Match them to the pairings using the unique inverse just proved. A count of five in both lists alone is not a proof of a bijection: the rules in both directions must undo one another.

### Problem 5

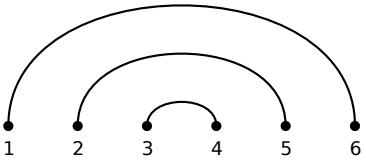
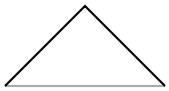
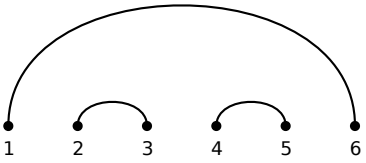
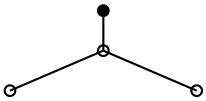
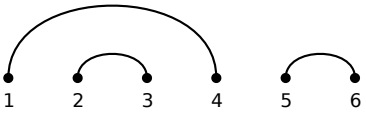
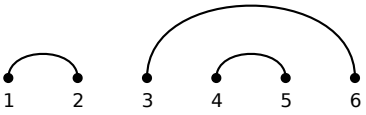
UUUDDUDD is valid: 18|25|34|67. UDDUUDUD is invalid: after its third letter the height is -1, so a closing endpoint has no opening partner. UUDUDUDD is valid: 18|23|45|67. UUUUDDUD is invalid: five U and three D leave height 2 and two unmatched openings. Hint: track height with counters; separate “went below zero” from “did not finish at zero.”

### Problem 6

Exactly 14 pairings, page 7. The four cases for the partner of 1 give 5+2+2+5. Within each case, choices inside and after the first pair are independent. The forced-split proof on page 9 justifies completeness. Hint: circle the pair touching 1 in every drawing and sort those drawings into groups.

## The five objects of size three

Complete key for K-1 P1-P2, grades 2-3 P1/P4 and grades 4-5 P1. Pairings are shown as arcs in fixed order; bend the row into the circle without changing partners. Each U adds height or opens a pair; in a tree it travels one edge away from the root.

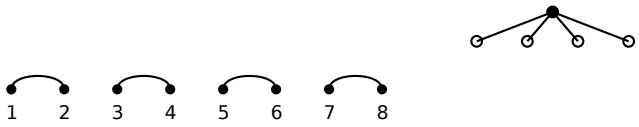
| | PAIRING | PATH | ORDERED TREE |
|---|---|---|---|
| **UUUDDD** |   |  |  |
| **UUDUDD** |   |  |  |
| **UUDDUD** |   |  |  |
| **UDUUDD** |   |  |  |
| **UDUDUD** |   |  |  |

For the three-edge trees, a second completeness proof sorts by the number of root children. One child leaves a two-edge tree below it: 2 cases. Two children distribute one extra edge to left or right: 2 cases. Three children are all leaves: 1 case. Total 2+2+1=5.

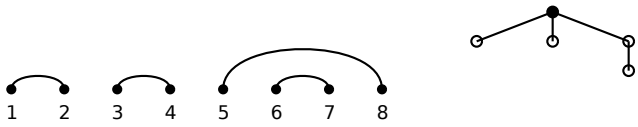
## The complete collection of size four

Fourteen eight-dot pairings and fourteen four-edge ordered trees. This answers K-1 P4, grades 2-3 P6 and grades 4-5 P5. Codes uniquely identify the drawings. For a completeness argument, group by the position of the first return to height zero.

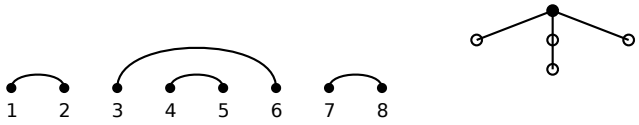
**UDUDUDUD**



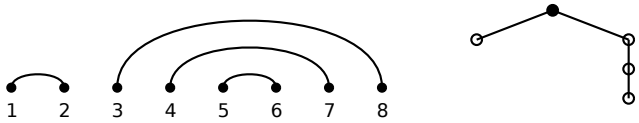

**UDUDUUDD**



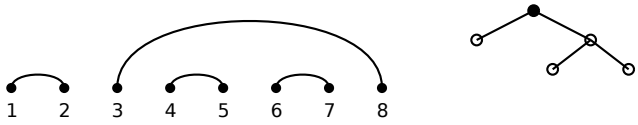

**UDUUDDUD**



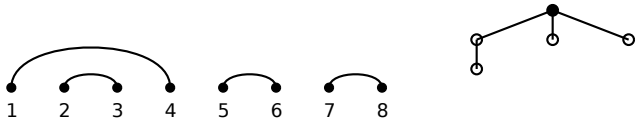

**UDUUUDDD**



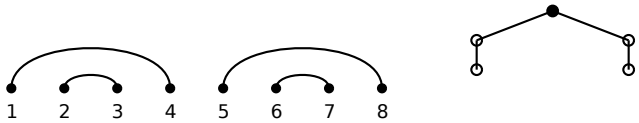

**UDUUDUDD**



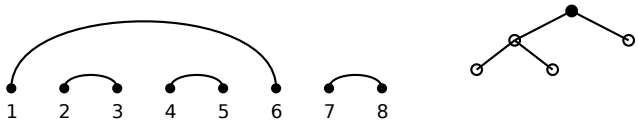

**UUDDUDUD**



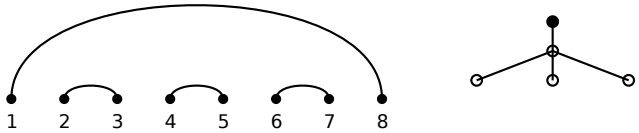

**UUDDUUDD**



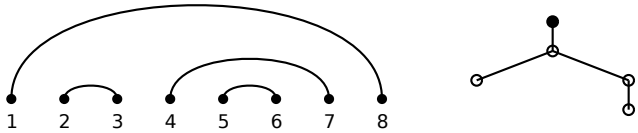

**UUDUDDUD**



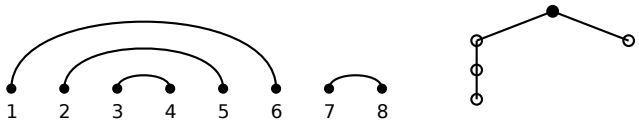

**UUDUDUDD**



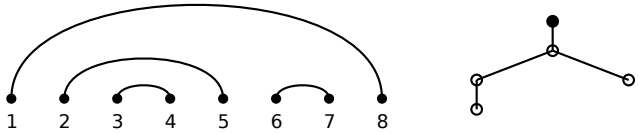

**UUDUUDDD**



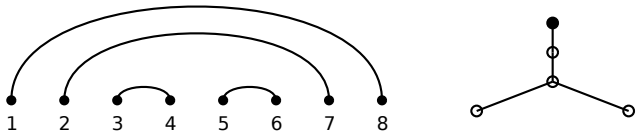

**UUUDDDUD**



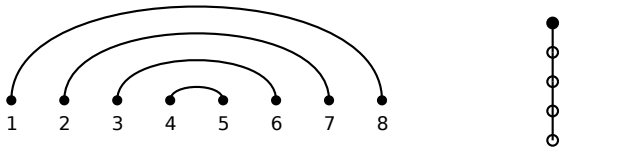

**UUUDDUDD**




**UUUDUDDD**




**UUUUDDDD**



# Grades 4-5 exact answers

## Problem 1

The five-edge example has two root children: the left has two leaf children, and the right has one. Exactly five three-edge trees; see page 6 and its root-child case split. A chain and a fork are different; changing edge lengths is irrelevant. Swapping unequal subtrees creates a different ordered tree. Hint: decide how many children the root has, then account for every remaining edge.

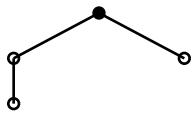
## Problem 2

The worked two-edge fork has code UDUD: visit the left leaf and return, then visit the right leaf and return. U is away from the root on the tree and upward on the path. For the four unanswered trees, reading top-left, top-right, bottom-left, bottom-right: UUUUDDDD, UDUDUDUD, UUDUDDUD, UDUUUDDD. Number of edges is half the code length. Maximum height is maximum root-to-vertex distance. A UD peak records a leaf. Every return to zero finishes a root child’s branch, so the number of such returns is the root’s number of children. Hint: use a finger as the walker; do not count a return as another new edge.

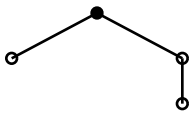
## Problem 3

No. For these ordered rooted trees, each legal complete code describes exactly one tree. UUDDUD: the root has two children; the left child has one child. UDUUDD: the root has two children; the right child has one child. UUDUDUDD: the root has one child, and that child has three leaf children. UUUDDUDD: the root has one child; that child has two children, the left of which has one child. Use the drawings below. At U there is one prescribed action: add a new rightmost child of the current vertex and move there. At D move to the parent. No choices remain, so there is exactly one ordered tree.

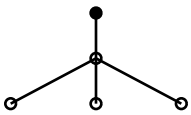
**UUDDUD**



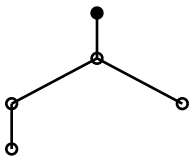
**UDUUDD**



**UUDUDUDD**



**UUUDDUDD**



## Problem 4

Valid: UUUUDDDD, UDUDUUDD, UUUDUDDD. Invalid: UUDDDUUD first goes below zero at letter 5; DUUUDDUD does so at letter 1; UUDUDUDU finishes at height 2. Necessary and sufficient rules: every prefix has at least as many U as D, and total U equals total D. The empty code gives the root-only tree. The construction from P3 never asks for a parent above the root and finishes at the root. Walking its new children left-to-right gives back the code, which proves sufficiency and reversibility.

# The exhaustive count and its proof

### Grades 4-5 Problem 5

There are 14 four-edge trees. The page 7 gallery exhibits them; this argument certifies none are missed. A nonempty legal code has exactly one split U P D Q, with the displayed D being the first return to height zero. P and Q are themselves legal codes. If P has i pairs, Q has n-1-i pairs. Choosing them independently gives C(i) times C(n-1-i) choices, and different i or different smaller words cannot give the same whole word.

### Grades 4-5 Problem 6

Let C(0)=1, the one empty code or root-only tree. For n at least 1, C(n) is the sum of C(i)C(n-1-i) for i=0,...,n-1. The values C(0) through C(6) are 1,1,2,5,14,42,132. In particular C(4)=5+2+2+5=14; C(5)=14+5+4+5+14=42; C(6)=42+14+10+10+14+42=132. This is a disjoint, exhaustive decomposition, not extrapolation from a number pattern.

In a tree, remove the edge from the root to its first child. P describes that child’s subtree; Q describes the remaining root branches. Conversely attach any P tree as the new leftmost child branch of any Q tree. These operations undo one another. In a pairing, 1 must meet an even endpoint, and the intervals inside and after this pair are independent smaller pairings. A child may give either concrete explanation without the formula.

Held hints: First ask where the walk returns to the root for the first time. Then cut the word just after that return. If needed, cover the initial U and its matching D and ask what remains. Give only as much of the split as the child needs.

# Extensions that preserve the mathematics

Compare maximum path height with deepest tree branch and deepest arc nesting. Sort the five size-three objects by number of peaks and leaves; these statistics agree because a leaf is an immediate U followed by D. Let crossings in for four points and locate why the inverse code stops being unique. Do not require a third representation in a single hour.

# Sources and fidelity limits

Mathematical objects: Richard P. Stanley, Exercises on Catalan and Related Numbers, Exercise 6.19 (shown as 19), parts (e), (i), (n), (o), printed pp. 221-223, https://math.mit.edu/~rstan/ec/catalan.pdf. Further university context: Alexander Postnikov, MIT 18.217 course sequence, September 17-24, 2021, https://math.mit.edu/~apost/courses/18.217_2021/. The solutions and small enumerations here are independently constructed for these exact final problems.

Teaching evidence: Natasha Rozhkovskaya, Math Circles for Elementary School Students, “Introduction: Berkeley 2009,” describes additional instructors and active parent help for individual attention. Stable tables, this launch, held hints and timing are local design proposals informed by the project’s concrete-first guidance; they have not been classroom tested. The guide covers ordered rooted plane trees, not arbitrary unlabeled trees, labeled trees or binary trees.

Verification: independent generation of all perfect matchings and an interleaving-endpoint filter checked counts through six pairs, all fixed-chord cases, neighbor inevitability and both inverse maps. Editable source and the full checked code lists are in facilitator-src. The revised student examples and physical-mat labels were checked against the source, and every final page was rendered for inspection.

# Starting routes and return visits

Planning update of 4 October 2026. The timing menus are examples; a theme can continue over several meetings. Keep the full Grades 4-5 investigations and choose by readiness.

### Use and return choices

On a return visit, K-1 can keep the six-dot collection and choose fixed-pair completions (P3) or the no-neighbor question (P6); the eight-dot collection (P4) can be its own investigation. Grades 2-3 can rebuild pairings from codes and decide which codes are legal (P3-5), then collect eight-dot pairings (P6). Grades 4-5 can return to inverse tree construction (P3-4) before the larger counts and recurrence (P5-6). Keep the first collection visible so a new representation has something concrete to describe.